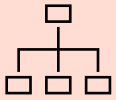
Tipos de datos recursivos

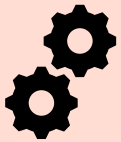
Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027

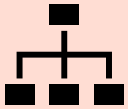
Tipos de datos recursivos



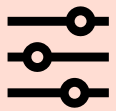
Un tipo recursivo es aquel que tiene alguna propiedad con valores del mismo tipo



Los tipos de datos recursivos dan lugar a algoritmos recursivos específicos



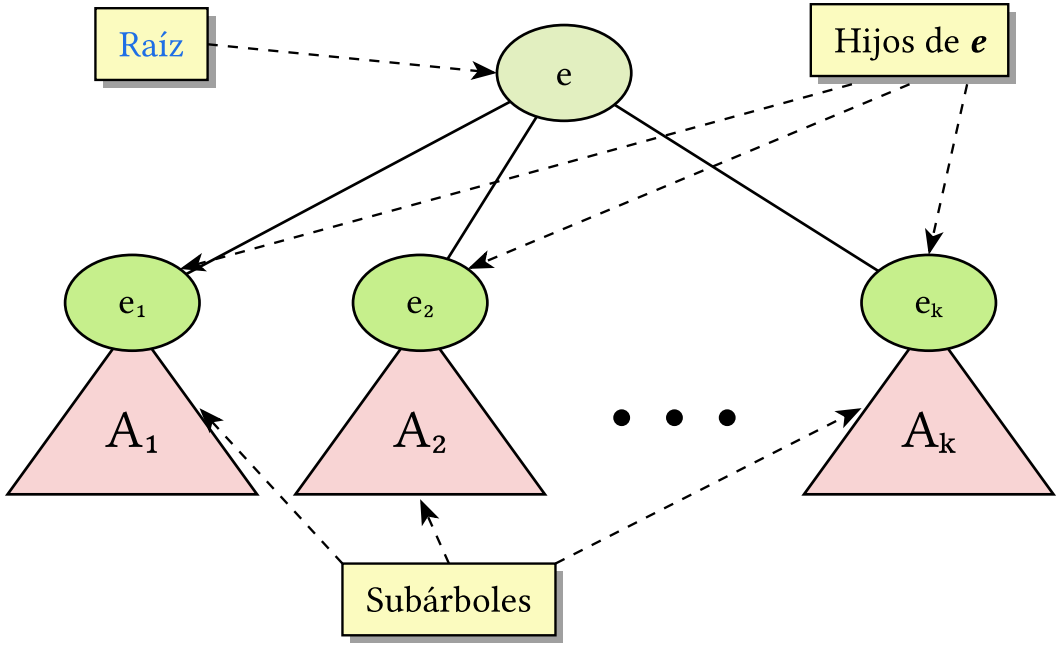
Asumimos que cada tipo recursivo tiene un conjunto de subtipos disjuntos



El tipo puede definir un conjunto de predicados que indiquen de qué subtipo es

Árbol

- Un árbol es un tipo de datos jerárquico, con una raíz y subárboles hijos
- Un árbol (Tree<E>) puede ser vacío, puede tener un elemento, que llamaremos etiqueta, o puede ser un árbol n-ario que tiene n hijos
- Si un árbol no tiene padre se llama raíz
- Veremos 2 tipos: árboles n-arios y árboles binarios



Árbol n-ario

Propiedades del tipo Tree:

```
sealed interface Tree<E> permits TEmpty<E>, TLeaf<E>, TNary<E>{
        static Tree<String> parse(String s){ … }
        static <R> Tree<R> empty() { … }
        static <R> Tree<R> leaf(R label) { … }
        static <R> Tree<R> nary(R label, List<Tree<R>> elements){…}
        int size();
        int height();
        Tree<E> copy();
        Tree<E> reverse();
        <R> Tree<R> map(Function<E, R> f);
        String toString();
        Stream<Tree<E>> byDepth();
        Stream<TreeLevel<E>> byLevel();
        …
}
```

Árbol binario

Propiedades del tipo BinaryTree:

```
sealed interface BinaryTree<E> permits BEmpty<E>, BLeaf<E>,
BTree<E>{
        static Tree<String> parse(String s){ … }
        static <R> BinaryTree<R> empty() { … }
        static <R> BinaryTree<R> leaf(R label) { … }
        static <R> BinaryTree<R> binary(R label,BinaryTree<E> left,
                BinaryTree<E> right){…}
        int size();
        int height();
        Tree<E> copy();
        <R> BinaryTree<R> map(Function<E, R> f);
        String toString();
        Stream<BinaryTree<E>> byDeph();
        Stream<BinaryTreeLevel<E>> byLevel();
        …
}
```

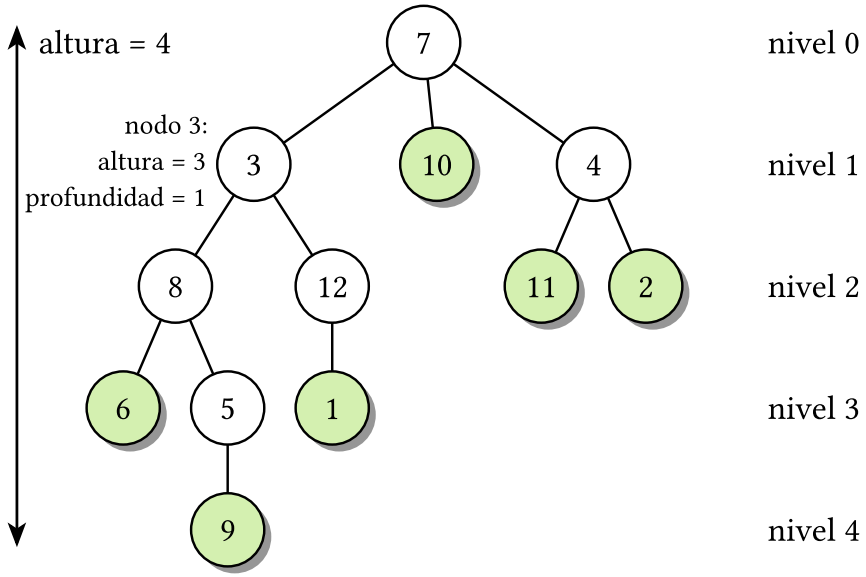
Nociones sobre árboles

- Podemos definir los predicados isEmpty(), isLeaf(), isNary() que serán true si el tipo del árbol es el adecuado → getType()
- Un árbol binario se dice **ordenado** si es vacío, es hoja, o si sus hijos están ordenados y su etiqueta es mayor que todas las de su hijo izquierdo y menor que las de su hijo derecho.
- Un árbol binario es **equilibrado** si es vacío, es hoja, o si sus hijos están equilibrados y sus alturas no difieren en más de una unidad.
- El **tamaño** de un árbol (size) es el número de etiquetas que tiene
- Un **camino** en un árbol es cualquier secuencia de árboles, t0, t1,…, tr−1, tal que cada uno es padre del siguiente. Entre cada padre y uno de sus hijos existe una arista
- La **longitud del camino** se define como el número de sus aristas

Árbol

- La **altura** de un árbol (height) se define como la longitud del camino más largo que comienza en la raíz y termina en una hoja

- La **profundidad** (depth) de un árbol se define como la longitud del camino que comienza en la raíz y termina en él
  - A la profundidad de un árbol también se la denomina nivel del árbol en el árbol que lo contiene

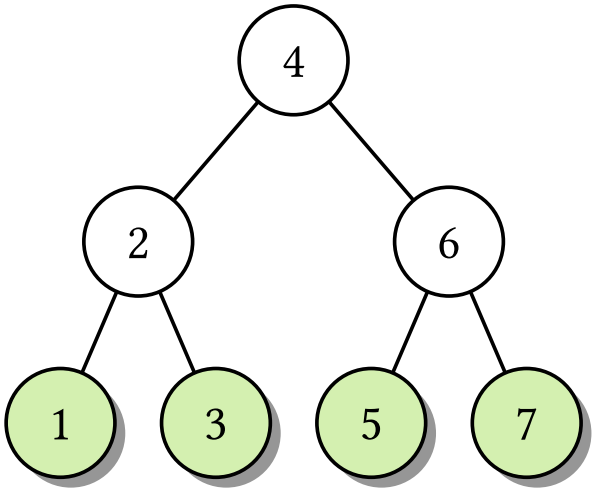


Recorridos árboles

Se pueden definir recursivamente de la siguiente forma:

Preorden	Se visita primero el árbol actual luego el primer hijo, posteriormente el segundo hijo, etc.
Postorden	Se visita primero el primer hijo, posteriormente el segundo hijo derecho, etc. y por último el árbol actual.
Inorden	Se visita primero el primer hijo, luego el árbol actual y finalmente el segundo hijo, etc.
General en profundidad	Es una generalización de los anteriores donde se indica la posición en el momento en el que se visitará el árbol actual.
En anchura o por niveles	Se visita primero el árbol actual (nivel 0), luego los árboles de nivel 1 de izquierda a derecha, etc.

Recorridos árboles



- **Preorden:** 4, 2, 1, 3, 6, 5, 7
- **Postorden:** 1, 3, 2, 5, 7, 6, 4
- **Inorden:** 1, 2, 3, 4, 5, 6, 7
- **En anchura:** 4, 2, 6, 1, 3, 5, 7

Árbol

- BinaryTree, es un tipo especial de árbol cuyos hijos se denominan hijo izquierdo y derecho

*Tipo **BinaryTree<E>**: Inmutable*

- *Métodos de creación:*
  - *BinaryTree<E> empty();*
  - *BinaryTree<E> leaf(E e);*
  - *BinaryTree<E> binary(E e, BinaryTree<E>> left, BinaryTree<E>> right);*
- *Propiedades:*
  - *BinaryTreeType getType();*
  - *Boolean isRoot();*
  - *BinaryTree<E> getFather(); Preconditions = !isRoot()*

- AVLTree<E> se utiliza para implementar un SortedSet<E>

- *Propiedades privadas*
  - *BinaryTee<E> tree;*
  - *Comparator<E> comparator;*
- *Invariant: tree está ordenado con respecto a comparator y equilibrado*
- *Operaciones privadas:*
  - *BinaryTree<E> add(BinaryTee<E> tree, Comparator<E> comparator);*
  - *BinaryTree<E> remove(BinaryTee<E> tree, Comparator<E> comparator);*
  - *BinaryTree<E> equilibrate(BinaryTee<E> tree);*
  - *Boolean isSorted(BinaryTree<E> tree, Comparator<E> comparator);*
  - *Boolean isEquilibrated(BinaryTree<E> tree);*

Ejemplos

Implementación: el código está estructurado alrededor de un switch sobre los valores de la propiedad type.

- **Tree**. Casos base: Empty y Leaf. Caso recursivo: árbol Nary.
- **BinaryTree**. Casos base: Empty y Leaf. Caso recursivo: Binary.

Ejemplo: tamaño de un árbol binario

```
public static <E> Integer size(BinaryTree<E> tree) {
   return switch(tree) {
      case BEmpty<E>    t -> 0;
      case BLeaf<E>  t -> 1;
      case BTree<E>  t -> 1 + size(t.left())  + size(t.right());
   };
}
```

Ejemplos

Implementación: el código está estructurado alrededor de un switch sobre los valores de la propiedad type.

- **Tree**. Casos base: Empty y Leaf. Caso recursivo: árbol Nary.
- **BinaryTree**. Casos base: Empty y Leaf. Caso recursivo: Binary.

Ejemplo: tamaño de un árbol n-ario

```
public static <E> Integer size(Tree<E> tree) {
   return switch(tree) {
      case TEmpty<E>    t -> 0;
      case TLeaf<E>  t -> 1;
      case TNary<E>  t -> 1 + t.children().stream().mapToInt(x->size(x)).sum();
   };
}
```

Ejemplo: altura de un árbol n-ario

```
public static <E> Integer height(Tree<E> tree) {
      return switch(tree) {
         case TEmpty<E> t -> -1;
         case TLeaf<E>  t -> 0;
         case TNary<E>  t -> 1 + t.children().stream().mapToInt(x->height(x)).max().getAsInt();
      };
}
```

Ejemplos

- Ejemplo: copia de un árbol binario

```
public static <E> BinaryTree<E> copy(BinaryTree<E> tree) {
   return switch (tree) {
      case BEmpty<E>  t -> BinaryTree.empty();
      case BLeaf<E>     t -> BinaryTree.leaf(t.label());
      case BTree<E>     t -> BinaryTree.binary(t.label(),copy(t.left()), copy(t.right()));
   };
}
```

- Ejemplo: copia de un árbol n-ario

```
public static <E> Tree<E> copy(Tree<E> tree) {
   return switch (tree) {
      case TEmpty<E>  t -> Tree.empty();
      case TLeaf<E>     t -> Tree.leaf(t.label());
      case TNary<E>     t -> Tree.nary(t.label(),t.children().stream().map(x->copy(x)).toList());
   };
}
```

Algoritmos sobre árboles

La complejidad de estos algoritmos se hace de la manera habitual asumiendo que el tamaño del árbol es el número de subárboles que tiene.

Algoritmos iterativos:

1. En profundidad: t.byDepth()
2. En anchura: t.byLevel()

Algoritmos recursivos: Al generalizar aparecen dos tipos de parámetros

3. **Parámetros de contexto** que guardan la ubicación en el árbol
4. **Parámetros acumuladores** para acumular el resultado. Son usualmente de un tipo mutable

Ejemplo:

Diseñe un algoritmo que dado un árbol n-ario de caracteres devuelva un conjunto de cadenas de caracteres que contenga todas las cadenas palíndromas que se formen desde la raíz a una hoja no vacía.

Código

Ejemplos

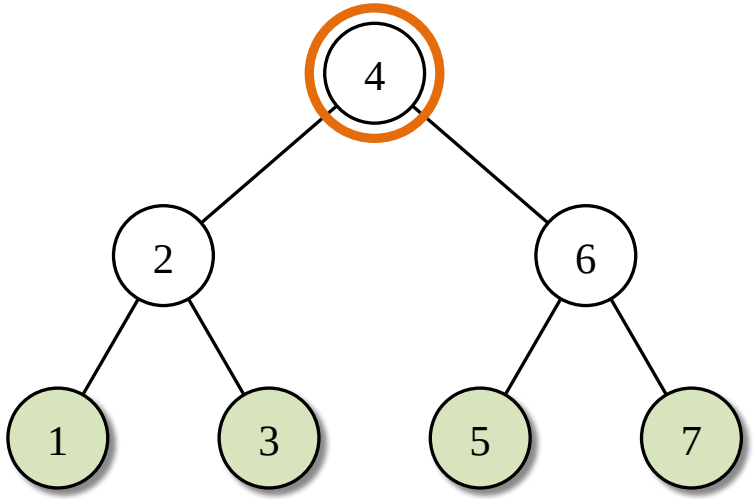
Recorrido en postorden de un árbol binario

```
public static <E> List<E> toListPostOrden(BinaryTree<E> tree) {
   return switch(tree) {
      case BEmpty() -> new ArrayList<>();
      case BLeaf(var lb) -> {
         List<E> r = new ArrayList<>();
         r.add(lb);
         yield r;
      }
      case BTree(var lb,var lt, var rt) -> {
         List<E> r = toListPostOrden(lt);
         r.addAll(toListPostOrden(rt));
         r.add(lb);
         yield r;
      }
   };
}
```

Recorrido en postorden de un árbol n-ario

```
public static <E> List<E> toListPostOrden(Tree<E> tree) {
   return switch(tree) {
      case TEmpty()  -> new ArrayList<>();
      case TLeaf(var lb) -> {
         List<E> r = new ArrayList<>();
         r.add(lb);
         yield r;
      }
      case TNary(var lb, var chd)  -> {
         List<E> r = chd.stream().flatMap(x->toListPostOrden(x).stream()).collect(Collectors.toList());
         r.add(lb);
         yield r;
      }
   };
}
```

Traza en postorden (paso 1/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
     List<E> r = toListPostOrden(lt);
     r.addAll(toListPostOrden(rt));
     r.add(lb);  // visita (al salir)
     yield r;
  }
 };
}
```

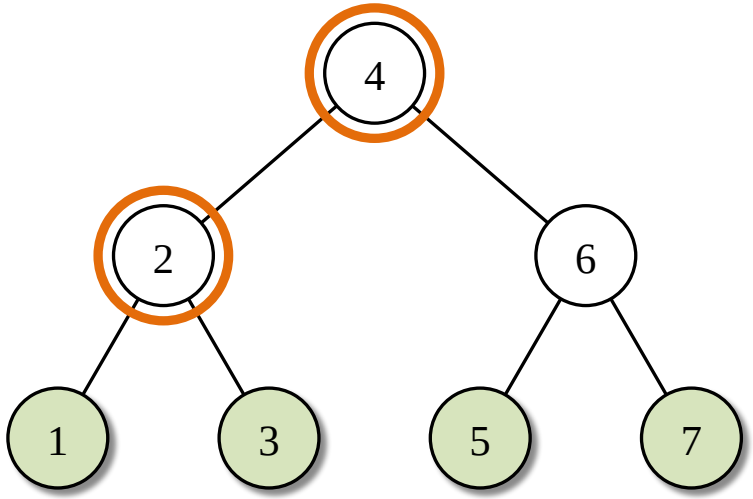
toListPostOrden(4): antes de añadir 4 hay que recorrer sus hijos

r =

llamada en curso: aún no se añade su etiqueta

visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 2/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
     List<E> r = toListPostOrden(lt);
     r.addAll(toListPostOrden(rt));
     r.add(lb);  // visita (al salir)
     yield r;
  }
 };
}
```

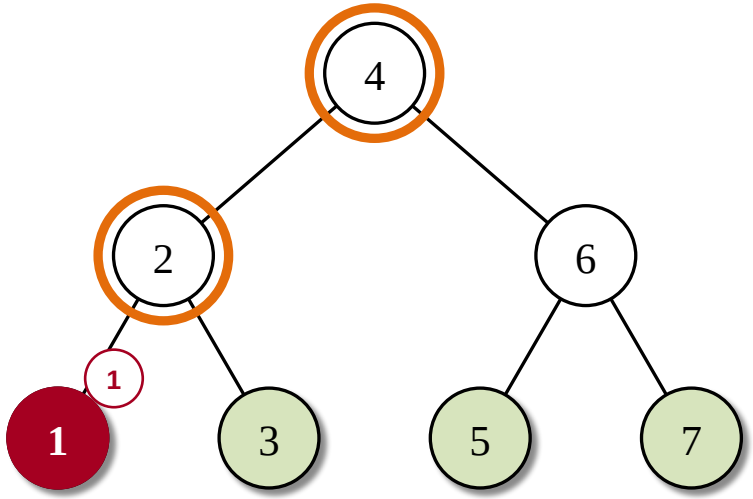
Primero el hijo izquierdo: toListPostOrden(2)

r =

llamada en curso: aún no se añade su etiqueta

visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 3/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

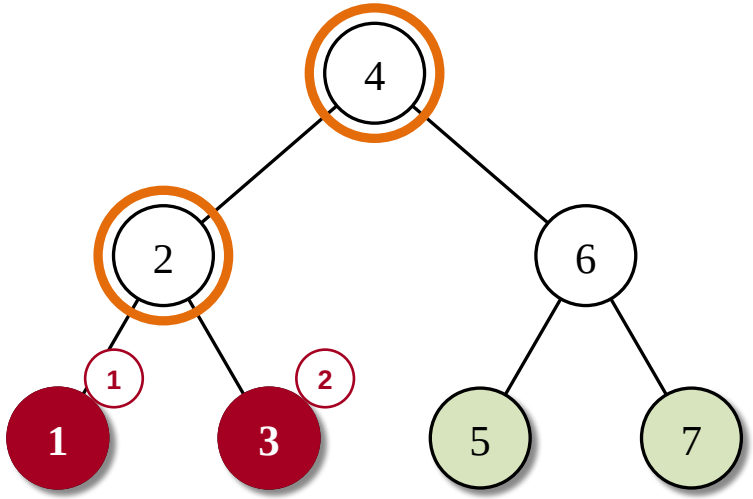
1 es hoja → devuelve [1]

r = | **1** | | | | | | |

○ llamada en curso: aún no se añade su etiqueta

● visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 4/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

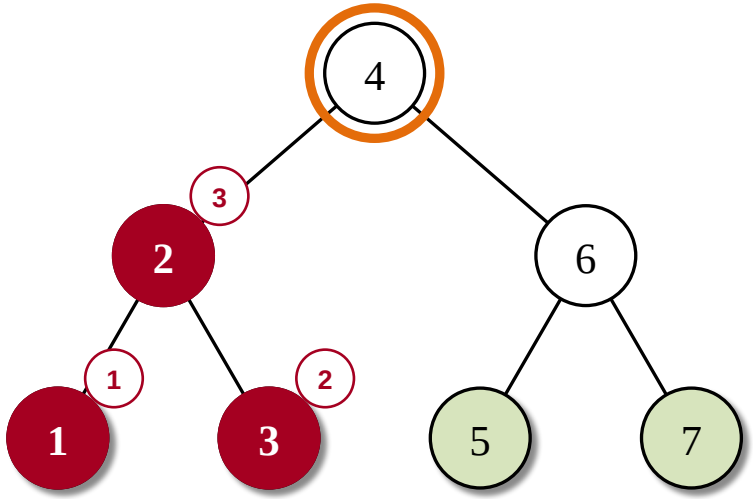
3 es hoja → devuelve [3]; addAll: r = [1, 3]

r = **1** **3**

○ llamada en curso: aún no se añade su etiqueta

● visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 5/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

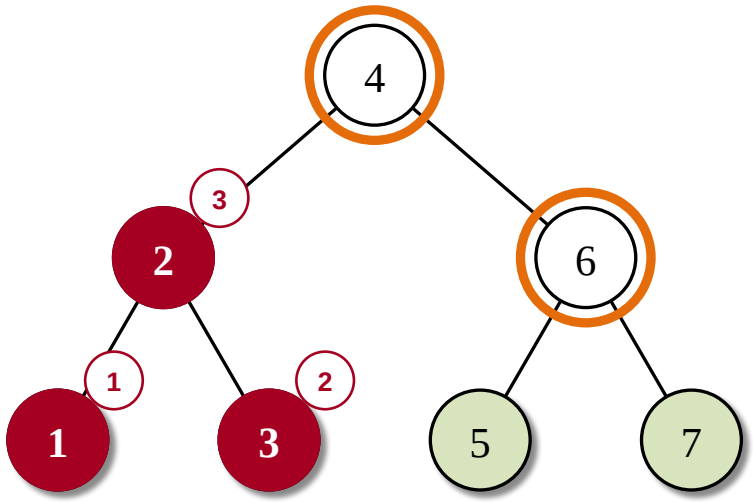
Fin de los hijos de 2: r.add(2) → devuelve [1, 3, 2]

r = 1 | 3 | 2 | | | | |

○ llamada en curso: aún no se añade su etiqueta

● visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 6/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

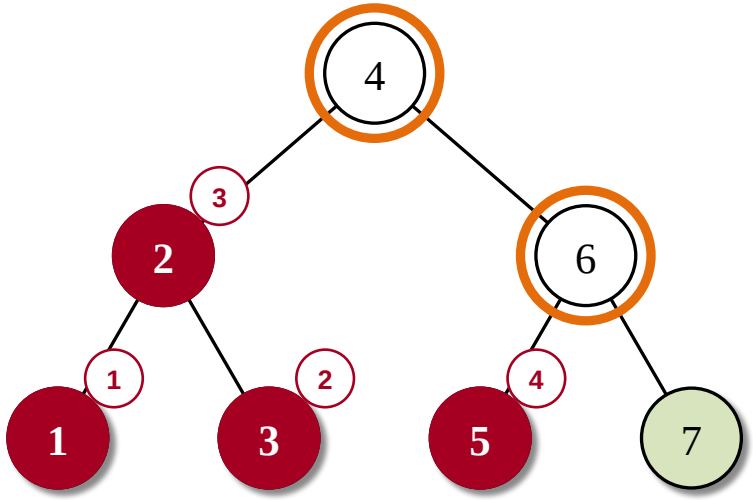
De vuelta en 4, ahora el hijo derecho: toListPostOrden(6)

r = 1 3 2

○ llamada en curso: aún no se añade su etiqueta

● visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 7/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

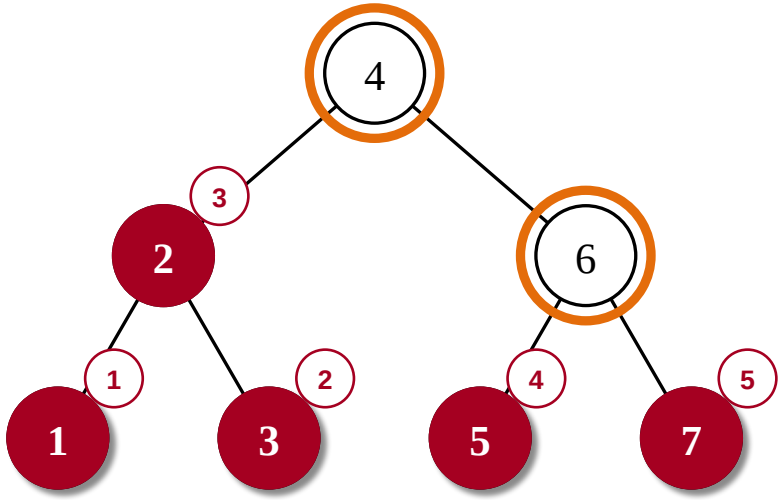
5 es hoja → devuelve [5]

r = | 1 | 3 | 2 | 5 | | | |

○ llamada en curso: aún no se añade su etiqueta

● visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 8/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

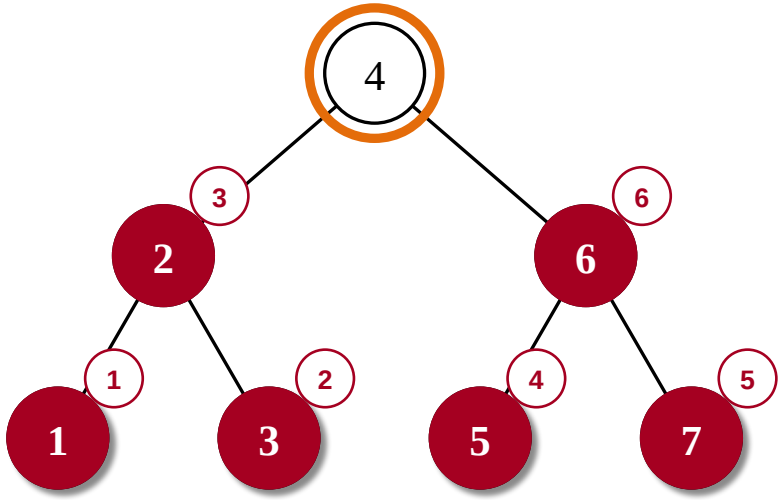
7 es hoja → devuelve [7]; addAll: r = [5, 7]

r = 1 3 2 5 7

llamada en curso: aún no se añade su etiqueta

visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 9/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

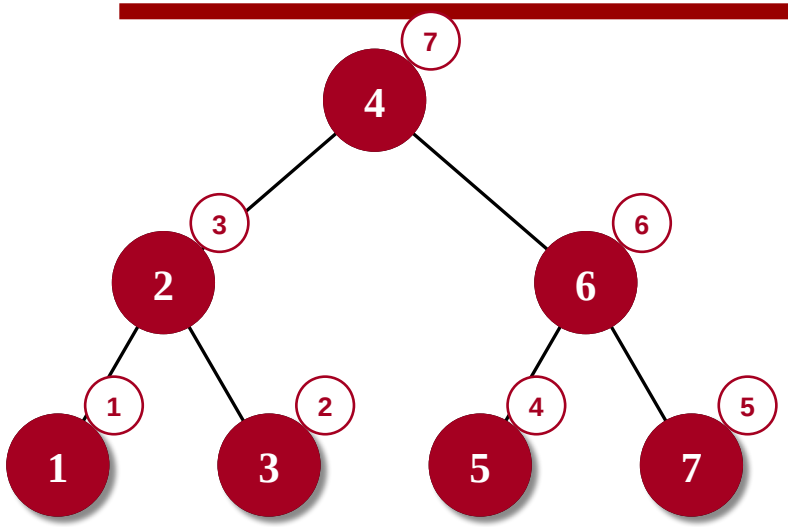
Fin de los hijos de 6: r.add(6) → devuelve [5, 7, 6]

r = 1 3 2 5 7 6

○ llamada en curso: aún no se añade su etiqueta

● visitado: etiqueta añadida a r (orden en la marca)

Traza en postorden (paso 10/10)




```
static <E> List<E> toListPostOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = toListPostOrden(lt);
    r.addAll(toListPostOrden(rt));
    r.add(lb);  // visita (al salir)
    yield r;
  }
 };
}
```

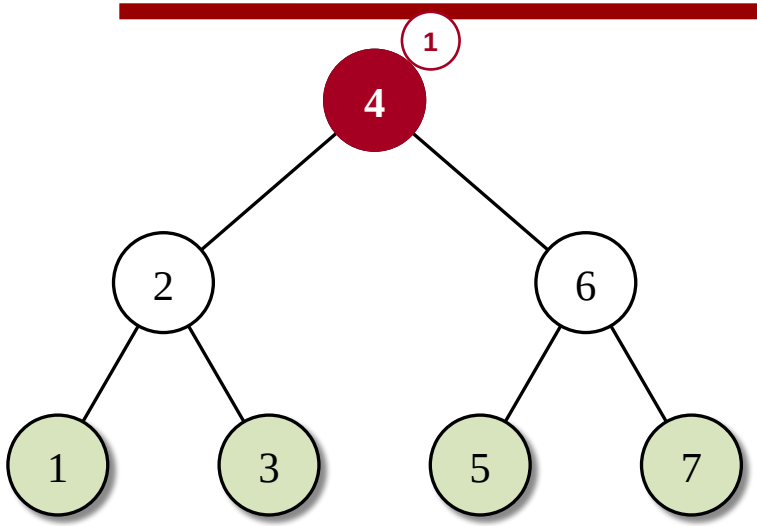
En 4: [1, 3, 2] + [5, 7, 6] y r.add(4) → [1, 3, 2, 5, 7, 6, 4]

r = 1 3 2 5 7 6 4

○ llamada en curso: aún no se añade su etiqueta

● visitado: etiqueta añadida a r (orden en la marca)

Traza en preorden (paso 1/7)



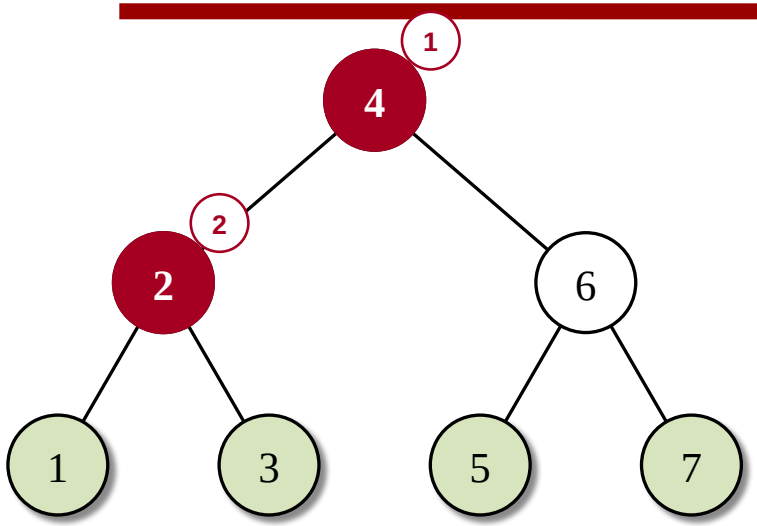

```
static <E> List<E> toListPreOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
     List<E> r = new ArrayList<>();
     r.add(lb);  // visita (al entrar)
     r.addAll(toListPreOrden(lt));
     r.addAll(toListPreOrden(rt));
     yield r;
  }
 };
}
```

toListPreOrden(4): r.add(4) nada más entrar → [4]

r = 

visitado: en preorden la etiqueta se añade nada más entrar en el nodo, antes de recorrer sus hijos

Traza en preorden (paso 2/7)



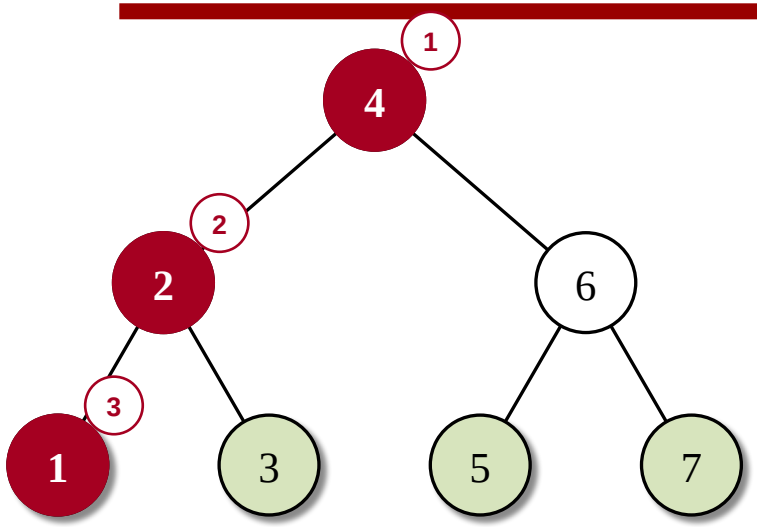

```
static <E> List<E> toListPreOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
     List<E> r = new ArrayList<>();
     r.add(lb);  // visita (al entrar)
     r.addAll(toListPreOrden(lt));
     r.addAll(toListPreOrden(rt));
     yield r;
  }
 };
}
```

Hijo izquierdo, toListPreOrden(2): r.add(2) → [4, 2]

r = [4] [2] [ ] [ ] [ ] [ ] [ ]

visitado: en preorden la etiqueta se añade nada más entrar en el nodo, antes de recorrer sus hijos

Traza en preorden (paso 3/7)



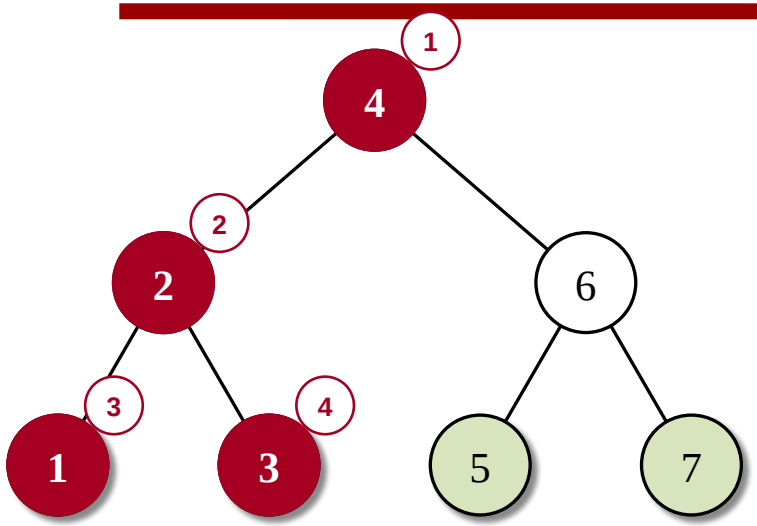

```
static <E> List<E> toListPreOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = new ArrayList<>();
    r.add(lb);  // visita (al entrar)
    r.addAll(toListPreOrden(lt));
    r.addAll(toListPreOrden(rt));
    yield r;
  }
 };
}
```

1 es hoja → [4, 2, 1]

r = 4 2 1

visitado: en preorden la etiqueta se añade nada más entrar en el nodo, antes de recorrer sus hijos

Traza en preorden (paso 4/7)



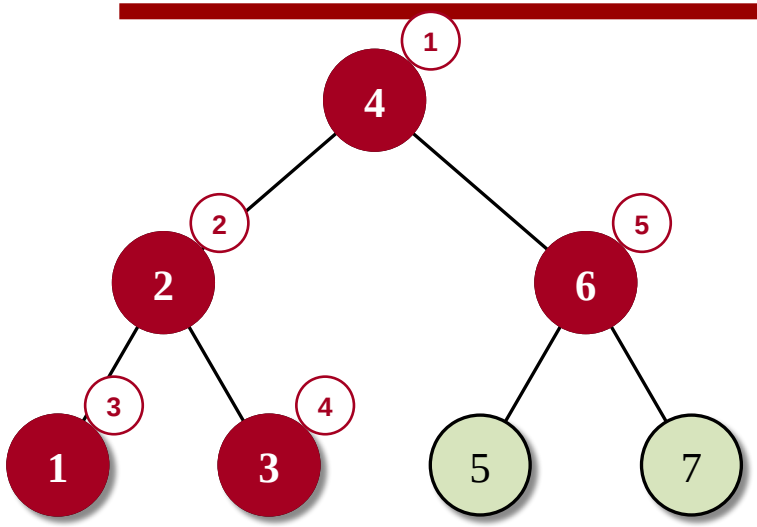

```
static <E> List<E> toListPreOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = new ArrayList<>();
    r.add(lb);  // visita (al entrar)
    r.addAll(toListPreOrden(lt));
    r.addAll(toListPreOrden(rt));
    yield r;
  }
 };
}
```

3 es hoja → [4, 2, 1, 3]; termina el subárbol de 2

r = 4 2 1 3

visitado: en preorden la etiqueta se añade nada más entrar en el nodo, antes de recorrer sus hijos

Traza en preorden (paso 5/7)



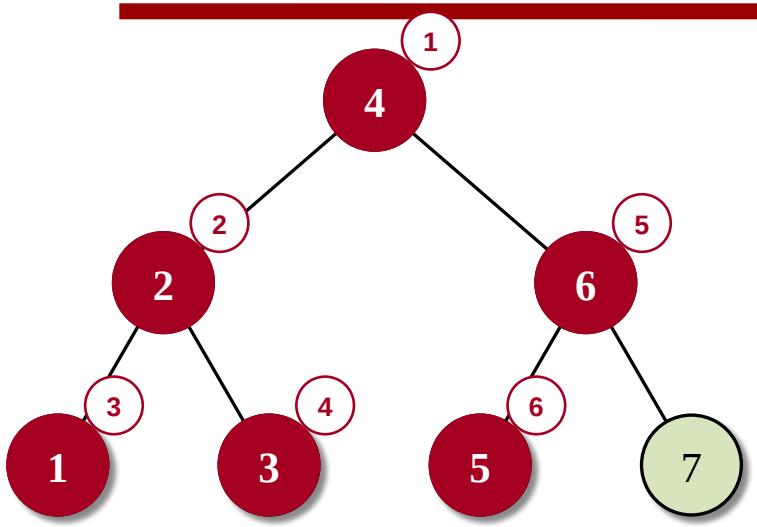

```
static <E> List<E> toListPreOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
     List<E> r = new ArrayList<>();
     r.add(lb);  // visita (al entrar)
     r.addAll(toListPreOrden(lt));
     r.addAll(toListPreOrden(rt));
     yield r;
  }
 };
}
```

Hijo derecho de 4, toListPreOrden(6): r.add(6) → [4, 2, 1, 3, 6]

r = 4 | 2 | 1 | 3 | 6

visitado: en preorden la etiqueta se añade nada más entrar en el nodo, antes de recorrer sus hijos

Traza en preorden (paso 6/7)



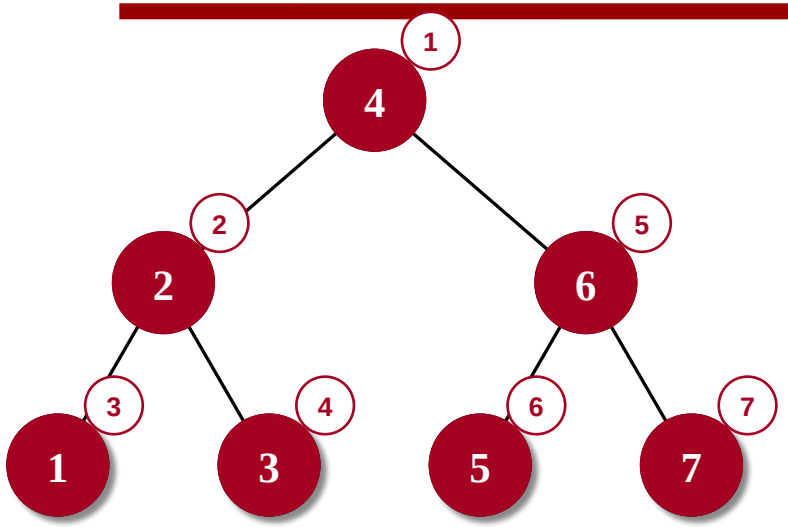

```
static <E> List<E> toListPreOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = new ArrayList<>();
    r.add(lb);  // visita (al entrar)
    r.addAll(toListPreOrden(lt));
    r.addAll(toListPreOrden(rt));
    yield r;
  }
 };
}
```

5 es hoja → [4, 2, 1, 3, 6, 5]

r = 4 2 1 3 6 5

visitado: en preorden la etiqueta se añade nada más entrar en el nodo, antes de recorrer sus hijos

Traza en preorden (paso 7/7)




```
static <E> List<E> toListPreOrden(
                   BinaryTree<E> tree) {
 return switch(tree) {
  case BEmpty() -> new ArrayList<>();
  case BLeaf(var lb) ->
      new ArrayList<>(List.of(lb));  // visita
  case BTree(var lb, var lt, var rt) -> {
    List<E> r = new ArrayList<>();
    r.add(lb);  // visita (al entrar)
    r.addAll(toListPreOrden(lt));
    r.addAll(toListPreOrden(rt));
    yield r;
  }
 };
}
```

7 es hoja → resultado final [4, 2, 1, 3, 6, 5, 7]

r = 4 2 1 3 6 5 7

● visitado: en preorden la etiqueta se añade nada más entrar en el nodo, antes de recorrer sus hijos

Iteradores sobre árboles

```
public static class DepthPathTree<E> implements Iterator<Tree<E>>, Iterable<Tree<E>> {

   public static <E> DepthPathTree<E> of(Tree<E> tree){
      return new DepthPathTree<E>(tree);
   }

   private Stack<Tree<E>> stack;

   public DepthPathTree(Tree<E> tree) {
      super();
      this.stack = new Stack<>();
      this.stack.add(tree);
   }

   public Iterator<Tree<E>> iterator() {
      return this;
   }

   public boolean hasNext() {
      return !this.stack.isEmpty();
   }

   public Tree<E> next() {
      Tree<E> actual = stack.pop();
      switch(actual) {
         case TNary<E> t:
            for(Tree<E> v:List2.reverse(t.children())) {
               stack.add(v);
            }
            break;
         case TEmpty<E> t:break;
         case TLeaf<E> t:break;
      }
      return actual;
   }
}
```

En profundidad (pila)

Problemas sobre árboles

Tres tipos fundamentales:

1. Problemas iterativos sin tener en cuenta el nivel de cada subárbol: se resuelven fácilmente usando stream o el iterador del árbol.

2. Problemas iterativos que tienen en cuenta el nivel de cada subárbol: se resuelven usando el stream byLevel() o el iterador equivalente proporcionado por el árbol.

3. Problemas recursivos: **Todos** los problemas de árboles pueden ser resueltos recursivamente.

Ejemplo: contar etiquetas de un árbol binario, versión iterativa.

```
public static <E> Integer numLabels(BinaryTree<E> tree) {
   return (int) tree.byDepth().filter(t -> !t.isEmpty())
               .mapToInt(t->1).count();
}
```

Más ejemplos

Contar etiquetas que cumplen un predicado:

```
public static <E> Integer countIfPredicate(BinaryTree<E> tree,
            Predicate<E> predicate) {
   return switch(tree) {
      case  BEmpty<E> t -> 0;
      case  BLeaf<E> t -> predicate.test(t.label())?1:0;
      case  BTree<E> t-> (predicate.test(t.label())?1:0)
            + countIfPredicate(t.left(), predicate)
            + countIfPredicate(t.right(),predicate);
   };
}
```

Más ejemplos

Comprobar si un árbol binario está ordenado, debemos hacerlo recursivamente:

```
public static <E> Boolean isOrdered(BinaryTree<E> tree, Comparator<E> cmp) {
   return switch (tree) {
      case BEmpty<E> t -> true;
      case BLeaf<E> t -> true;
      case BTree<E> t -> {
         Optional<E> lfLabel = maxLabelOrdered(t.left(), cmp);
         Optional<E> rgLabel = minLabelOrdered(t.right(), cmp);
         Boolean r = (lfLabel.isEmpty() || cmp.compare(lfLabel.get(), t.label()) < 0)
               && (rgLabel.isEmpty() || cmp.compare(rgLabel.get(), t.label()) > 0);
         yield r && isOrdered(t.left(), cmp) && isOrdered(t.right(), cmp);
      }
   };
}
```

Más ejemplos

Etiqueta más pequeña con respecto a un orden, árbol **no ordenado**:

```
public static <E> Optional<E> minLabelRecursive(BinaryTree<E> tree,
            Comparator<Optional<E>> cmp) {
   return switch (tree) {
      case BEmpty<E>  t -> Optional.empty();
      case BLeaf<E>  t -> Optional.of(t.label());
      case BTree<E>  t -> {
         Optional<E> e1 = minLabelRecursive(t.left(), cmp);
         Optional<E> e2 = minLabelRecursive(t.right(), cmp);
         yield Stream.of(Optional.of(t.label()),e1, e2)
         .filter(e->!e.isEmpty()).min(cmp).get();
      }
   };
}
```

Versión iterativa:

```
public static <E> Optional<E> minLabel(BinaryTree<E> tree, Comparator<E> cmp) {
   return tree.byDepth().map(tt -> tt.optionalLabel())
         .filter(x-> x.isPresent()).map(x->x.get()).min(cmp);
}
```

Más ejemplos

Mismo problema, con el árbol **ordenado** (**recursivo** es más eficiente):

```
public static <E> Optional<E> minLabelOrdered(BinaryTree<E> tree, Comparator<E> cmp) {
   return switch (tree) {
      case BEmpty<E>  t -> Optional.empty();
      case BLeaf<E>  t ->  Optional.of(t.label());
      case BTree<E>  t when t.left() instanceof BEmpty<E>  tl ->
                        Optional.of(t.label());
      case BTree<E>  t when t.left() instanceof BLeaf<E>  tl ->
                        Optional.of(tl.label());
      case BTree<E>  t when t.left() instanceof BTree<E>  tl ->
                    minLabelOrdered(tl, cmp);
      default -> throw new IllegalArgumentException("Tipo de entrada inesperado: "
               + tree.getClass().toString());
   };
}
```

Más ejemplos

Comprobar si un árbol (suponemos no ordenado) contiene la etiqueta label:

```
public static <E> Boolean containsLabelRecursive(BinaryTree<E> tree, E label) {
   return switch (tree) {
      case BEmpty<E>  t -> false;
      case BLeaf<E>  t -> t.label().equals(label);
      case BTree<E>  t -> t.label().equals(label)
            || containsLabel(t.left(), label)
            || containsLabel(t.right(), label);
   };
}
```

Versión iterativa:

```
public static <E> Boolean containsLabel(BinaryTree<E> tree, E label) {
   return tree.byDepth().map(t -> t.optionalLabel())
      .filter(e -> e.isPresent()).map(e->e.get())
      .anyMatch(e -> e.equals(label));
}
```

Más ejemplos

Comprobar si un árbol (suponemos ordenado) contiene la etiqueta label:

```
public static <E> Boolean containsLabelOrdered(BinaryTree<E> tree,
         E label, Comparator<E> cmp) {
   return switch (tree) {
      case BEmpty<E> t -> false;
      case BLeaf<E> t -> t.label().equals(label);
      case BTree<E> t when cmp.compare(label, t.label()) == 0
               -> true;
      case BTree<E> t when cmp.compare(label, t.label()) < 0
               -> containsLabelOrdered(t.left(), label, cmp);
      case BTree<E> t when cmp.compare(label, t.label()) > 0
               -> containsLabelOrdered(t.right(), label, cmp);
      default -> throw new IllegalArgumentException("Tipo de entrada inesperado: "
               + tree.getClass().toString());
   };
}
```

Ejercicios

1. Diseñe un algoritmo que dado un árbol binario de enteros devuelva cierto en caso de que para cada nodo que tenga 2 hijos no vacíos se cumpla que su etiqueta es igual a la suma de las etiquetas de las raíces de sus 2 hijos.
2. Implemente una función booleana que, dados un árbol binario de caracteres y una lista de caracteres, determine si existe un camino en el árbol de la raíz a una hoja que sea igual a la lista.
3. Diseñe un algoritmo que dado un árbol n-ario Tree<E> y un predicado sobre Tree<E> devuelva una lista List<Boolean> de forma que el elemento i-ésimo de la lista será true si todos los árboles del nivel i cumplen el predicado.
4. Diseñe un algoritmo que dado un árbol n-ario de tipo genérico E devuelva un Map<Integer,List<E>> que incluya una entrada en el map por cada nivel del árbol. Dicha entrada debe tener como clave el nivel y como información asociada una lista con las etiquetas de los nodos que se encuentran en dicho nivel y que tienen un número par de hijos.
5. Diseñe un algoritmo recursivo que dado un árbol n-ario de tipo genérico devuelva un Map<Integer, List<Tree<E>>>. Dicho map en cada entrada tiene como clave el número de hijos de un árbol y como valor una lista con todos los árboles que tienen ese número de hijos.

Tipos de datos recursivos

Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2026-2027